



北京師範大學
BEIJING NORMAL UNIVERSITY

计算方法 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 7 月 2 日

目录

1 环境说明	3
2 问题描述	3
3 三种数值积分方法	3
3.1 Euler 方法	3
3.1.1 原理	3
3.1.2 实现	3
3.2 改进 Euler 方法	3
3.2.1 原理	3
3.2.2 实现	4
3.3 Runge-Kutta 四阶方法	4
3.3.1 原理	4
3.3.2 实现	4
4 收敛阶计算	4
5 实验结果	4
5.1 结果表格	5
5.2 收敛阶估计	5

1 环境说明

本文全部代码为 c++ 代码, 硬件为 Macbook Air M2, 软件为 Xcode。在 Xcode 内部进行基于命令行的编译运行。

本实验报告使用 $\text{\LaTeX}$ 进行制作, 基于 overleaf 网站进行在线写作。

2 问题描述

编制 Euler 方法、修正 Euler 方法、4 阶 Runger-Kutter 方法的通用程序;

对于初值问题: $y' = -x^2y^2$, $0 \leq x \leq 1.5$, $y(0) = 3$ 。利用上述程序计算, 通过取不同的步长 h 验证方法的收敛阶。其中精确解为: $y = \frac{3}{1+x^3}$ 。

3 三种数值积分方法

3.1 Euler 方法

3.1.1 原理

Euler 方法基于泰勒展开的一阶截断, 近似公式为:

$$y_{n+1} = y_n + hf(x_n, y_n)$$

3.1.2 实现

```
while (x < x_end) {  
    y += h * f(x, y);  
    x += h;  
}
```

3.2 改进 Euler 方法

3.2.1 原理

该方法采用梯形公式进行修正:

$$\begin{aligned}k_1 &= f(x_n, y_n) \\k_2 &= f(x_n + h, y_n + hk_1) \\y_{n+1} &= y_n + \frac{h}{2}(k_1 + k_2)\end{aligned}$$

3.2.2 实现

```
double k1 = f(x, y);
double k2 = f(x + h, y + h * k1);
y += h / 2 * (k1 + k2);
```

3.3 Runge-Kutta 四阶方法

3.3.1 原理

四阶 Runge-Kutta 方法具有较高精度，形式如下：

$$\begin{aligned} k_1 &= f(x_n, y_n) \\ k_2 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\ k_4 &= f(x_n + h, y_n + hk_3) \\ y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

3.3.2 实现

```
double k1 = f(x, y);
double k2 = f(x + h/2, y + h/2 * k1);
double k3 = f(x + h/2, y + h/2 * k2);
double k4 = f(x + h, y + h * k3);
y += h / 6 * (k1 + 2*k2 + 2*k3 + k4);
```

4 收敛阶计算

设 $E(h)$ 为步长 h 对应的误差，数值方法的收敛阶 p 满足：

$$E(h) \approx Ch^p$$

因此，通过两组误差 $E(h), E(h/2)$ 可估算：

$$p \approx \log_2 \left(\frac{E(h)}{E(h/2)} \right)$$

5 实验结果

我们在 $x = 1.5$ 处分别使用 $h = 0.1875, 0.09375, 0.046875$ 进行计算，并比较误差。结果如下表。

5.1 结果表格

方法	步长 h	数值解 $y(1.5)$	误差
Euler	0.1875	0.6271578379	0.0585564478
Euler	0.09375	0.6605859065	0.0251283792
Euler	0.046875	0.6740206221	0.0116936636
Improved Euler	0.1875	0.7063956414	0.0206813557
Improved Euler	0.09375	0.6902677634	0.0045534777
Improved Euler	0.046875	0.6867760357	0.0010617500
Runge-Kutta 4	0.1875	0.6859560784	0.0002417927
Runge-Kutta 4	0.09375	0.6857279292	0.0000136435
Runge-Kutta 4	0.046875	0.6857150874	0.0000008017

表 1: 不同方法在 $x = 1.5$ 的数值解与误差

5.2 收敛阶估计

Euler 方法收敛阶计算结果为 1.1036, 约为 1, 验证了其为一阶方法。

改进 Euler 方法收敛阶计算结果为 2.1005, 约为 2, 具有更高精度。

Runge-Kutta 四阶方法收敛阶计算结果为 4.0891, 约为 4, 误差最小。

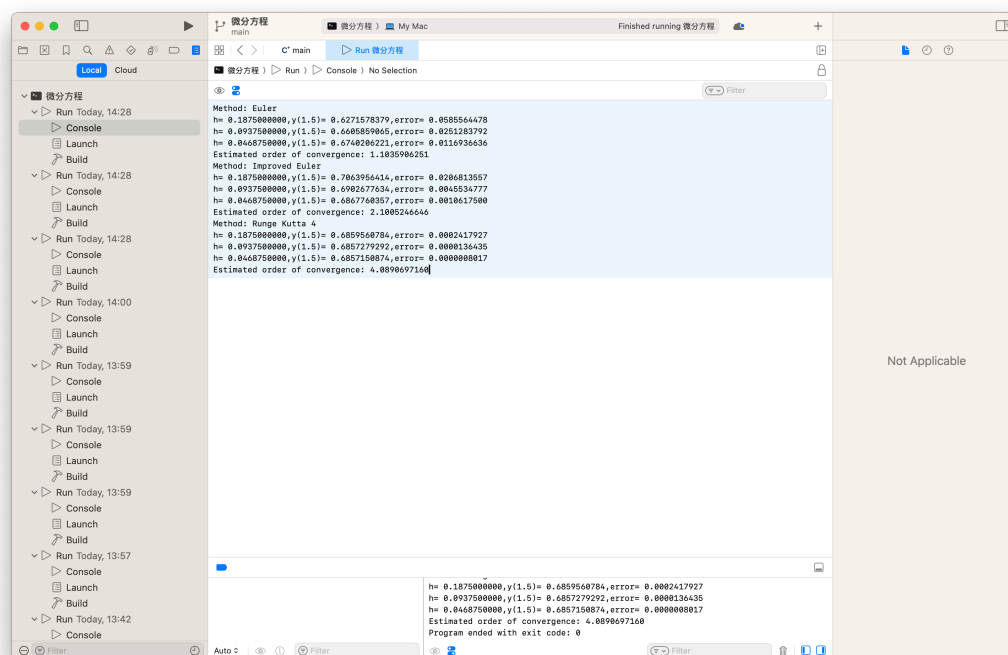


图 1: 输出结果